POLITECNICO
MILANO 1863

AN2DL - First Challenge Report
marcusai

Amirali Askari, Hamidreza Khodadadi,
Zahra Nazar Zadeh Attar, Mohammad Rashid Khan Mofrad

November 17, 2025

1 Introduction

This report presents the complete workflow for the *Pirate Pain Intensity Classification Challenge*, which involves predicting one of three levels **no_pain**, **low_pain**, and **high_pain** from 160-timestep sequences of 3D skeletal joint measurements.

The goal is to design temporal deep learning architectures capable of modelling human motion dynamics while being robust to class imbalance and noisy joint channels. A major challenge is that the high_pain class is significantly underrepresented, making it difficult for models to learn discriminative temporal features.

Two different models are explored in this work. The first one is a carefully engineered architecture combining **Bidirectional LSTMs (BiLSTMs)**, **Multi-Head Self-Attention (MHA)**, and a **Swish-activated dense block**, trained with oversampling and hyperparameter tuning across several learning rates and batch sizes. The second model is a more complex architecture that integrates **1D-CNN layers**, stacked **BiLSTMs**, a custom **attention mechanism**, careful temporal padding, and **test-time augmentation (TTA)** to better capture local motion patterns and improve generalization.

2 Problem Analysis

The dataset contains time-series data showing how body joints move over time. Each sample includes 31 joint features recorded across 160 fixed timesteps. Every sequence is given a pain label based on human evaluation. By looking closely at the data, we can identify a few important characteristics:

2.1 Class Imbalance

The dataset shows a strong imbalance in the number of samples for each pain level. The **no_pain** class appears very often and makes up most of the data. The **low_pain** class has far fewer samples, and the **high_pain** class is extremely rare. This imbalance creates a challenge during training because the model may focus too much on the majority class and fail to learn the minority classes properly.

2.2 Feature Sparsity and Correlation

Some joint features show very small changes over time or are strongly correlated with other joints, which means they do not add much useful information. A few joints also contain sudden spikes or noise that can negatively affect training. Since some joints, like joint_30, were almost constant in all samples, they were removed because they did not provide any meaningful contribution to the model.

2.3 Metadata and Auxiliary Inputs

The dataset contains a small amount of metadata, including four survey values. In the first model, we did not use these survey features and relied only on the joint trajectories. However, in the second model, the survey data is used in a separate branch that processes these values in parallel with the motion data. This allows the model to combine both sources of information before the final prediction.

3 Method

3.1 Model 1: BiLSTM-MHA

The first model is built with three main parts. First, a two-stage BiLSTM encoder learns temporal patterns by processing the sequence forward and backward. Then, Layer Normalization and Dropout are used after each layer to stabilize training. Third, a Multi-Head Self-Attention block captures long-range dependencies between timesteps, supported by a residual connection and LayerNorm. Finally, a Swish-activated dense head performs the classification after temporal pooling.

The model is trained with sparse categorical cross-entropy, and class imbalance is handled through oversampling and monitoring macro-F1. Several learning rates and batch sizes are tested, and the best configuration is retrained on the full dataset using oversampled data.

3.2 Model 2: BiLSTM-CNN

The second model uses a more complex architecture designed to capture both local and global motion patterns. It begins with a series of 1D-CNN layers that extract short-range spatial–temporal features from the joint sequences. These are followed by stacked BiLSTM layers, which learn longer temporal dependencies. An attention mechanism is then applied to highlight the most informative timesteps.

In addition to the motion branch, a second branch processes the four survey features, and both branches are merged before the final dense layers. The model also includes padding strategies and uses test-time augmentation (TTA) to improve robustness. Training follows the same loss function as the first model.

4 Experiments

4.1 First model

Below is a summary of the explored configurations for model 1:

Table 1: Hyperparameter Configurations

Name	Learning Rate	Batch Size
lr1e-3_bs32_osTrue	1e-3	32
lr7e-4_bs32_osTrue	7e-4	32
lr5e-4_bs32_osTrue	5e-4	32
lr1e-3_bs16_osTrue	1e-3	16
lr7e-4_bs16_osTrue	7e-4	16
lr5e-4_bs16_osTrue	5e-4	16

Training behavior was monitored using accuracy, loss, and macro-F1 through a custom callback. Models converged quickly, typically before epoch 50.

The configuration **lr1e-3_bs32_osTrue** achieved the highest macro-F1 on the validation set and was selected as the final architecture.

4.2 Second model

The custom attention layer learns a weight for each timestep by applying a tanh transformation followed by a softmax over the sequence, so more relevant frames receive higher weights. The final context vector is a weighted sum of the inputs, allowing the model to focus on informative moments of the motion.

For the second model, we also build hand-crafted features. For every joint and each survey value, we compute statistics such as mean, standard deviation, minimum, maximum, quartiles, skewness, kurtosis, signal energy, absolute differences, and zero-crossing rate. These statistics turn each sequence into a fixed-length feature vector that summarizes both motion dynamics and survey information.

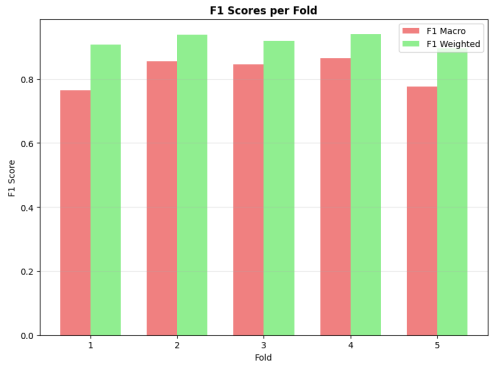


Figure 1: F1 scores of model 2

The F1 scores across all five folds show stable performance, with macro F1 remaining consistently high and weighted F1 slightly higher due to the class imbalance. This indicates that the model generalizes well across folds and handles minority classes reasonably effectively. The small variation between folds suggests that the training process is reliable and not overly sensitive to data splits.

5 Results

5.1 Validation Performance

The best model achieved:

- **First model F1:** ≈ 0.94, depending on random seed
- **Second model F1:** ≈ 0.95

Although accuracy is not particularly informative for imbalanced tasks, macro-F1 highlights the contribution of minority class performance.

5.2 Confusion Matrix

A typical confusion matrix shows that high_pain remains the hardest class, but oversampling significantly improves its recall compared to using raw data.

Table 2: Example Confusion Matrix for first model (Validation)

	Pred HP	Pred LP	Pred NP
HP	8	1	2
LP	2	17	0
NP	2	1	100

Final Model

these 2 models are the best models among all the models we tested and all the hyperparameter tunings that we done.

6 Discussion

Strengths

Both models show good performance but for different reasons. The first model works well thanks to its BiLSTMs and attention layer, which capture important temporal patterns. Oversampling also improves recall for the high_pain class, and the Swish dense layers help stabilize training.

The second model is stronger in feature learning. Its CNN blocks extract local motion details, the BiLSTMs capture longer dependencies. The custom attention layer highlights key timesteps, and the extra survey branch provides useful metadata that the first model does not use.

Weaknesses

The main limitation in both models is the noisy joint data, which can reduce accuracy. The high_pain class remains difficult to predict due to very few samples. The first model cannot use metadata, while the second model is more complex and requires more careful regularization to avoid overfitting.

7 Conclusion

We developed two temporal deep learning models for pain classification. The first combines BiLSTMs and Multi-Head Attention, while the second adds CNNs, a custom attention layer, and a metadata branch. After hyperparameter tuning and oversampling, both models were trained on the full dataset and used to generate Kaggle submissions.

Future work may focus on transformer-based models, better motion augmentation on unlabeled data.